

Mode decomposition

Currently I'm basing my functions as of the f-func paper:

In this section, we will expand on the derivation of Eq. (1) in some detail. Our starting point is the generic decomposition of a GW-signal template $h(\theta)$ as

$$h(\theta) = \sum_{m=1}^M h_m(\theta) + \epsilon(\theta), \quad (2)$$

with the following assumptions, valid for all m, m' :

$$\langle h_m | h_m \rangle > 1, \quad |\langle h_m | h_{m'} \rangle| \ll 1, \quad \langle \epsilon | \epsilon \rangle \ll 1, \quad (3)$$

$$\rho(\theta) := \sqrt{\langle h(\theta) | h(\theta) \rangle}.$$

$$X(\theta) := \langle x | \hat{h}(\theta) \rangle := \frac{\langle x | h(\theta) \rangle}{\rho(\theta)}.$$

Here, I assume that x and h are generated with the 'full modes'. However, FEW defaults to a mode selection algorithm which will rank modes based on their power and select the top/dominant ones up to a certain threshold. **I am still using that method for now.**

My mode decomposition consists of picking $M=7$ mode groups, which will help in computing these quantities:

$$[\rho(\theta)]_m := \rho_m(\theta) := \sqrt{\langle h_m(\theta) | h_m(\theta) \rangle},$$

$$[X(\theta)]_m := X_m(\theta) := \langle x | \hat{h}_m(\theta) \rangle := \frac{\langle x | h_m(\theta) \rangle}{\rho_m(\theta)}.$$

Where it iterates through each mode group m from the M selected modes.

$$h_n = \sum_m h_{2mn}$$

I say mode groups, because what I've selected is . So I only pick quadrupole modes (ell=2), and sum over all m for a certain n . I pick $n=\{-1,0,1,2,3,4,5\}$. Hence, 7 mode groups. It's a bit confusing so here's how it looks like in reality:

```

groups = [
    [(2,-2,-1),(2,-1,-1), (2,0,-1), (2,1,-1), (2,2,-1)],#0
    [(2,-2,0),(2,-1,0), (2,0,0), (2,1,0), (2,2,0)],#1
    [(2,-2,1),(2,-1,1), (2,0,1), (2,1,1), (2,2,1)],#2
    [(2,-2,2),(2,-1,2), (2,0,2), (2,1,2), (2,2,2)],#3
    [(2,-2,3),(2,-1,3), (2,0,3), (2,1,3), (2,2,3)],#4
    [(2,-2,4),(2,-1,4), (2,0,4), (2,1,4), (2,2,4)],#5
    [(2,-2,5),(2,-1,5), (2,0,5), (2,1,5), (2,2,5)],#6
]

[33]

amp = getattr(waveform_gen_sep.waveform_generator, 'amplitude_generator', None)
mode_forms = []

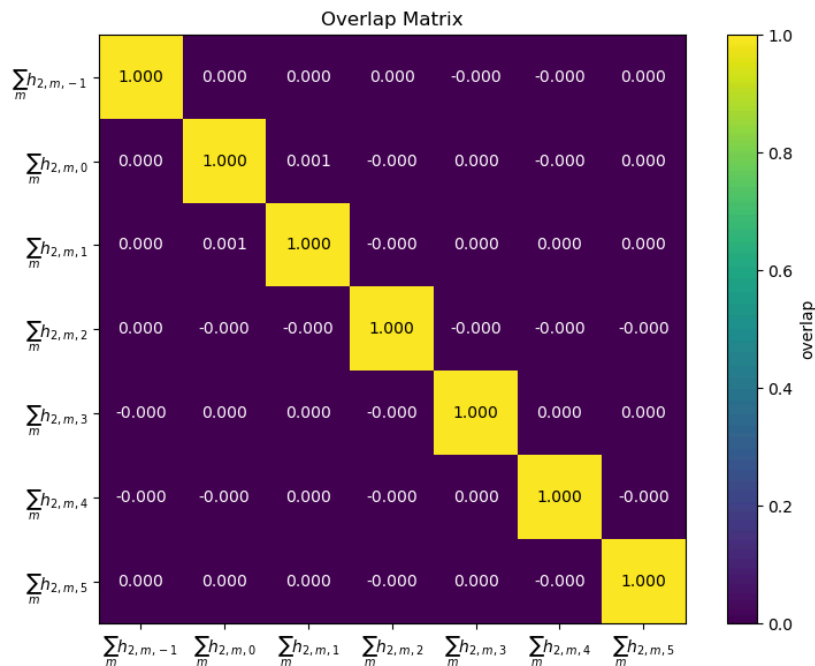
for point in line_points_proc1.T:
    logm1, logm2, a, p0, e0 = point
    m1 = 10**logm1
    m2 = 10**logm2

    # Generate waveforms for each GROUP
    waveforms_per_group = []
    for group in groups:
        mode_form= waveform_gen_comb(m1, m2, a, p0, e0, xI0, dist,
                                     qS, phiS, qK, phiK,
                                     Phi_phi0, Phi_theta0, Phi_r0,
                                     dt=dt, T=T,
                                     mode_selection=group, # Pass group list
                                     include_minus_mkn=False)

[34]

```

So for example, [(2,-2,-1),(2,-1,-1), (2,0,-1), (2,1,-1), (2,2,-1)] is one mode group.
 These are all orthogonal (based on their overlap):



$$\chi^2(\theta) := |\mathbf{X}(\theta) - \boldsymbol{\rho}(\theta)|^2$$

They are both relevant to compute the chi2 influence the exponential suppression of the peaks.

which will

Search

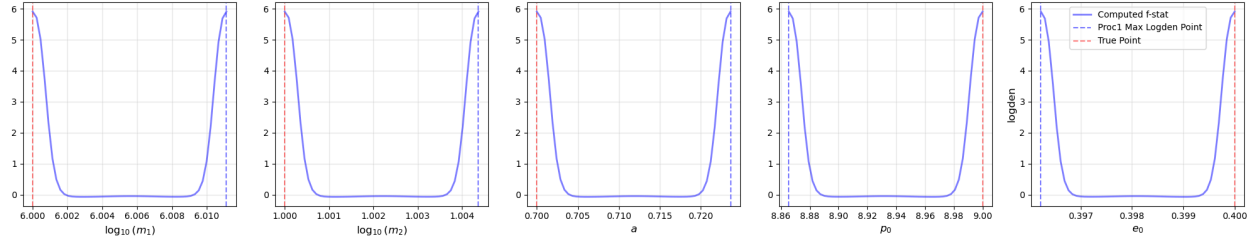
So far our efforts for the initial stage, PARIS1 stage, and PARIS2 stage are all okay. In those three initial stages, we use the f-function with timemax and phasemax.

Now, the problem is in getting the actual 2 year peak in the final stage (PARIS3), which is recommended to use the f-func but turning off timemax and phasemax.

The pipeline is working right now: using the posteriors from PARIS2 as the priors for PARIS3. The systematics should be working ok.

However, when trying to do optimization with ffunc(non-timemax,non-phasemax), I'm running across problems where it is still getting caught into secondaries.

For example, here's one with PARIS (3sigma ellipsoid). The original starting point was 6.01487994 1.00500901 0.73205648 8.81696887 0.39543265 (PARIS2 max ld point)



Here's a table of the true point vs secondary:

value	true	secondary
Physical point (logm1, logm2, a, p0, e0)	6, 1, 0.7, 9 0.4	6.01112701, 1.00437234, 0.72360392, 8.86546466, 0.39622541
Log_density (f-function, non-timemax, non-phasemax)	5.90716261	5.90755207
Opt SNR (rho)	5.942134448355127	5.98645821
X	5.94213445	5.93663066

Furthermore, here's a comparison of the opt SNR (rho_m) per mode group:

```

    rhostats_proc1[0] #sec
[67]
... array([0.19603076, 0.33790044, 1.54655541, 3.70011075, 3.19758276,
          1.98739979, 1.10101111])

    rhostats_proc1[-1] #true
[36]
... array([0.19248603, 0.32069291, 1.50244027, 3.64528079, 3.19028491,
          2.00740266, 1.12572087])

```

And the detection SNR per mode group (X_m):

```

    Xm_proc1[0] #sec
[38]
... array([0.19688621+0.j, 0.33278623+0.j, 1.57012775+0.j, 3.83140209+0.j,
          3.37847919+0.j, 2.13702537+0.j, 1.20192901+0.j])

    Xm_proc1[-1] #true
[39]
... array([0.19856525+0.j, 0.33403565+0.j, 1.57260846+0.j, 3.83283232+0.j,
          3.37960972+0.j, 2.14049363+0.j, 1.20711229+0.j])

```

I have two hypothesis (unsure if related or not):

First Hypothesis

I'm thinking that, since the secondary had a higher optimal SNR overall, then it should show in the mode groups (given if they are orthogonal), since the total SNR is a squared sum of the opt SNR per mode groups:

$$\rho^2 = \sum_m \rho_m^2$$

Hence even though $X_m(\text{true}) > X_m(\text{sec})$, but since $\rho_m(\text{sec}) > \rho_m(\text{true})$ due to higher SNR overall, then the chi2 suppression is bigger for the true instead of the secondary.

Here I did a test for the true point:

```
==  
rhom: 0.19248603078306445  
X: 0.198565245152033  
diff 3.695684734387368e-05  
==  
rhom: 0.32069290575123394  
X: 0.334035651624827  
diff 0.00017802886744728506  
==  
rhom: 1.5024402667889176  
X: 1.5726084559456244  
diff 0.004923574769531379  
==  
rhom: 3.6452807891788863  
X: 3.832832321197844  
diff 0.03517557716265815  
==  
rhom: 3.1902849110294134  
X: 3.3796097175839814  
diff 0.035843882376924624  
==  
rhom: 2.0074026584888958  
X: 2.140493631330451  
diff 0.017713207051911536  
==  
rhom: 1.125720873406202  
X: 1.2071122850892955  
diff 0.00662456189576679  
==  
chi2 0.0029040223055490358
```

And for the secondary point:

```

===
rhom: 0.19603075970169498
X: 0.19688621254784353
diff 7.31799571983649e-07
===
rhom: 0.3379004382785215
X: 0.3327862333150281
diff 2.6155092408620604e-05
===
rhom: 1.5465554118362044
X: 1.570127746213109
diff 0.0005556549479765954
===
rhom: 3.7001107480090836
X: 3.831402091352711
diff 0.017237416836974245
===
rhom: 3.197582763253977
X: 3.3784791930766285
diff 0.032723518322581455
===
rhom: 1.9873997918557642
X: 2.137025369320248
diff 0.022387813431580274
===
rhom: 1.1010111057745589
X: 1.2019290107321052
diff 0.010184423541020364
===
chi2 0.0019732033007756637

```

This reminds me of this paragraph:

nant mode is μ_M (which is often μ_{2200} , in large regions of the signal space). One of the key findings in Chua & Cutler [9] is that the strongest secondary peaks in the likelihood (or detection statistic) over the model parameter space are overwhelmingly caused by the dominant mode of a non-local template being phase-aligned with the dominant mode of the actual signal, *without significant contribution from any alignment of the other modes*.

So I'm thinking that this secondary is aligned with all the mode groups that I've selected?
Hence, maybe **missing modes**?

Second Hypothesis

I was also thinking that there are missing modes when we are computing the signal x and templates h , when evaluating the opt SNR and detection SNR. I've relied on the default FEW mode generation for it, since I think the paper implies we need all/most of the modes for both.

However, I was thinking if the modes problem could also arise from x and h . I tried if I only generate the signal x and the waveform templates h consistently with only the $(2,m,n)$ modes for $n=\{-1$ to $5\}$. For example, I tried to generate the signal with it:

```
modes_l2
[42]
... [(2, -2, -1),
      (2, -2, 0),
      (2, -2, 1),
      (2, -2, 2),
      (2, -2, 3),
      (2, -2, 4),
      (2, -2, 5),
      (2, -1, -1),
      (2, -1, 0),
      (2, -1, 1),
      (2, -1, 2),
      (2, -1, 3),
      (2, -1, 4),
      (2, -1, 5),
      (2, 0, -1),
      (2, 0, 0),
      (2, 0, 1),
      (2, 0, 2),
      (2, 0, 3),
      (2, 0, 4),
      (2, 0, 5),
      (2, 1, -1),
      (2, 1, 0),
      (2, 1, 1),
      (2, 1, 2),
      ...
      (2, 2, 1),
      (2, 2, 2),
      (2, 2, 3),
      (2, 2, 4),
      (2, 2, 5)]

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

h_dataset1 = waveform_gen_comb([
    10**logm1, 10**logm2, a, p0, e0,
    xI0, dist, qS, phiS, qK, phiK, Phi_phi0, Phi_theta0, Phi_r0,
    dt=dt, T=T,
    mode_selection = modes_l2,
    include_minus_mkn=False
])
[43]
```

Which gives me 88% of the power compared to the waveform originally generated with FEW mode selection:

```
[44]: # only n=-1 to 5
      gwf.rhostat(h_datasel)**2/gwf.rhostat(data)**2*100
...  array(88.24393532)
```

Now the chi2 is higher for the secondary:

```
====
rhom: 0.19603075970169498
X: 0.19281915212614115
diff 1.0314423219354762e-05
====
rhom: 0.3379004382785215
X: 0.3194241442514939
diff 0.000341373440973176
====
rhom: 1.5465554118362044
X: 1.4999613813321142
diff 0.002171003678616085
====
rhom: 3.7001107480090836
X: 3.6434357797644426
diff 0.003212052025531068
====
rhom: 3.197582763253977
X: 3.189089378798893
diff 7.213757950186002e-05
====
rhom: 1.9873997918557642
X: 2.0045587891681422
diff 0.0002944311887661962
====
rhom: 1.1010111057745589
X: 1.1216385134135953
diff 0.00042548994590698043
====
chi2 1.542011265017361e-05
```

While its lower for the true point (which we want):


```
====  
rhom: 0.19248603078306445  
X: 0.1944781149159225  
diff 3.968399192384861e-06  
====  
rhom: 0.32069290575123394  
X: 0.320693306544209  
diff 1.606350088782831e-13  
====  
rhom: 1.5024402667889176  
X: 1.502600622928083  
diff 2.5714091368042615e-08  
====  
rhom: 3.6452807891788863  
X: 3.6451429179258663  
diff 1.9008482409306366e-08  
====  
rhom: 3.1902849110294134  
X: 3.1899116942476655  
diff 1.3929076617821684e-07  
====  
rhom: 2.0074026584888958  
X: 2.007016277665154  
diff 1.4929014095527596e-07  
====  
rhom: 1.125720873406202  
X: 1.1254899340413687  
diff 5.333299022966661e-08  
====  
chi2 1.579374855859503e-11
```

However I don't think generating the signal with only certain modes is also what we want. So I'm a bit confused. I think it's a problem with the modes, but I'm not sure if it's the first or second hypothesis.